Staploy file Language Specification

Choiman1559

Update Date: Aug 31, 2026
Version Based on Writing: 1.0.0 (93a8302)

Contents

1 Introduction **5**
1.1 Purpose . . . 5
1.2 Scope . . . 5
1.3 Terminology . . . 6

2 Language Overview **7**
2.1 Staploy file . . . 7
2.2 HCL Foundation . . . 7
2.3 Execution Model . . . 8

3 File Syntax **8**
3.1 HCL Syntax . . . 8
3.2 Blocks . . . 9
3.3 Labels . . . 9
3.4 Attributes . . . 10
3.5 Expression . . . 10

4 Global Blocks **11**
4.1 `configure` . . . 12
4.2 `alias` . . . 12
4.3 `build` . . . 13
4.4 `manage` . . . 13
4.5 `target` . . . 14

5 Alias **15**
5.1 Alias Model . . . 15
5.2 Worker Alias . . . 16
5.3 App Alias . . . 17
5.4 Alias Resolution . . . 18

6 Worker Expressions **18**
6.1 Worker ID . . . 18
6.2 Worker Name . . . 19
6.3 `group:` Expression . . . 19
6.4 `alias:` Expression . . . 20
6.5 `where` Block . . . 20
6.5.1 Worker Attributes . . . 21
6.5.2 Exact Matching . . . 22
6.5.3 Prefix Matching . . . 22
6.5.4 Suffix Matching . . . 22
6.5.5 Regular Expression Matching . . . 22
6.5.6 Numeric Range Matching . . . 23
6.5.7 Negation . . . 23
6.5.8 Filter Selection . . . 24
6.5.9 Combined Conditions . . . 24

7 Application References **25**
7.1 Application Name . . . 25
7.2 Application Version . . . 26
7.3 `alias:` Expression . . . 26
7.4 `shell:` Expression . . . 26

8 Build **27**
8.1 Build Block . . . 27
8.2 Build Attributes . . . 28
8.2.1 `output_dir` . . . 28
8.2.2 `version` . . . 28
8.2.3 `executable` . . . 29
8.2.4 `envs` . . . 29
8.2.5 `lib_version` . . . 30
8.3 Target Architectures . . . 30
8.3.1 `share` Target . . . 31
8.4 Pre-build Commands . . . 32
8.5 Build Output . . . 32
8.6 Build Alias Resolution . . . 33

9 Manage Operations **34**
9.1 Manage Block . . . 34
9.2 `create` . . . 35
9.3 `upload` . . . 35
9.4 `delete` . . . 36
9.5 `pull` . . . 36

10 Target Operations **37**
10.1 Target Block . . . 37
10.2 Worker Selection . . . 38
10.3 `unset` . . . 39
10.4 `remove` . . . 39
10.5 `deploy` . . . 40
10.6 `push_only` . . . 41
10.7 `set_active` . . . 41
10.8 `disconnect` . . . 42

11 Shell Hooks **42**
11.1 Hook Model . . . 42
11.2 `pre_deploy` . . . 43
11.3 `post_deploy` . . . 43
11.4 Shell Execution . . . 44

12 Execution Semantics **45**
12.1 File Processing . . . 46
12.2 Alias Processing . . . 46
12.3 Build Processing . . . 46
12.4 Manage Processing . . . 47
12.5 Target Processing . . . 47

13 Resolution Semantics **48**
13.1 Worker Resolution . . . 48
13.2 Application Resolution . . . 49
13.3 Alias Resolution . . . 49
13.4 Build Artifact Resolution . . . 50
13.5 Resolution Constraints . . . 51

14 Error Semantics **52**
14.1 Syntax Errors . . . 52
14.2 Resolution Errors . . . 52
14.3 Build Errors . . . 52
14.4 Execution Errors . . . 53
14.5 Partial Failure . . . 53

15 Conformance **54**
15.1 Valid Staploy files . . . 54
15.2 Implementation Requirements . . . 55

16 Examples **55**
16.1 Minimal Staploy file . . . 56
16.2 Multi-architecture Build . . . 56
16.3 Worker Alias . . . 57

16.4 App Alias 57
16.5 Build and Upload 58
16.6 Build, Upload, and Deploy 59
16.7 Complete Deployment Example 60

1 Introduction

1.1 Purpose

Staploy file is a declarative configuration language used to describe application builds, package management, and deployment operations in the Staploy system. This specification defines the syntax and semantics of the Staploy file language. It describes the structure of a Staploy file, the meaning of its blocks and attributes, the resolution of aliases and other expressions, and the order in which operations are executed. The purpose of this specification is to provide a precise reference for users writing Staploy files and for implementations processing them. Examples in this document are intended to illustrate the language syntax and execution semantics rather than prescribe a particular deployment environment.

1.2 Scope

This specification covers the following aspects of the Staploy file language:

- File and block structure based on HCL syntax.
- Global configuration through the `configure` block.
- Worker and application aliases through the `alias` block.
- Worker selection and filtering expressions.
- Multi-architecture application builds through the `build` block.
- Application management operations through the `manage` block.
- Deployment operations through the `target` block.
- Shell hooks executed before and after deployment operations.
- Alias and application reference resolution.
- Operation ordering and execution semantics.
- Errors encountered during parsing, resolution, building, and execution.

This specification does not define the Staploy server protocol, worker implementation, package archive format, or the internal implementation details of the Staploy CLI, except where those details are observable through the Staploy file language.

1.3 Terminology

The following terms are used throughout this specification.

Staploy file A configuration file written in the Staploy file language. It describes builds, package management operations, and deployment targets.

Worker A Staploy worker process registered with a Staploy server. Workers are the execution targets for deployment operations.

Worker ID The unique identifier assigned to a worker.

Worker Alias A named expression representing one or more workers. A worker alias may contain explicit worker references, worker groups, and filtering conditions.

Application A deployable package managed by Staploy. An application is identified by its name and, where applicable, its version.

App Alias A named reference to an application name and optionally a version. App aliases may additionally be associated with the output of a build.

Build An operation that produces a Staploy application package from one or more architecture-specific build outputs.

Manage Operation An operation that manages an application package or its registration with the Staploy server, such as creating, uploading, deleting, or pulling a package.

Target A named deployment definition specifying a set of workers and one or more operations to perform on those workers.

Target Operation An operation performed against the workers selected by a target. Target operations include `unset`, `remove`, `deploy`, `push_only`, `set_active`, and `disconnect`.

Shell Hook A shell command executed as part of a target operation. Shell hooks may be executed before or after the main operation.

Alias Resolution The process of converting an alias expression such as `alias:name` into the concrete worker or application reference represented by that alias.

Worker Expression A string expression used to identify or select workers, including worker IDs, worker names, `group:` expressions, and `alias:` expressions.

2 Language Overview

2.1 Staploy file

A Staploy file is a declarative configuration file that describes a sequence of application build, package management, and deployment operations. It consists of a set of top-level blocks. The principal blocks are `configure`, `alias`, `build`, `manage`, and `target`. Each block defines a distinct part of the desired build or deployment workflow. This does not directly describe the internal requests sent between the Staploy CLI, server, and workers. Instead, it describes the desired operations at a higher level. The Staploy CLI parses the file, resolves references and aliases, and translates the resulting declarations into executable operations.

For example, a target may refer to a group of workers using a worker expression and may refer to an application using an application alias:

```
target "deploy_app" {
   workers = ["alias:production"]
   deploy "alias:my-app" {
      post_deploy = "app --version"
   }
}
```

The concrete workers, application name, and application version are resolved by the CLI before the corresponding operation is executed.

2.2 HCL Foundation

Staploy file uses HCL (HashiCorp Configuration Language) as its syntactic foundation. Consequently, Staploy file inherits the basic HCL concepts of blocks, labels, attributes, expressions, and collections. The use of HCL provides a human-readable representation while allowing the Staploy file language to remain structured and extensible. Staploy file defines its own semantics on top of HCL. HCL syntax alone does not determine the meaning of a Staploy file.

For example, `alias:production` and `group:all` are Staploy-specific worker expressions interpreted by the Staploy CLI.
Accordingly, this specification distinguishes between the syntactic rules provided by HCL and the semantic rules defined by Staploy file. Unless otherwise specified, constructs described in this document are interpreted according to the Staploy file semantics.

2.3 Execution Model

The processing of a Staploy file consists of several conceptual stages. The CLI first parses the file into its corresponding configuration objects. It then initializes the execution context and resolves references required by subsequent operations.

The general processing order is:

1. Parse the Staploy file.
2. Apply the global configuration.
3. Synchronize worker information with the Staploy server.
4. Process worker and application aliases.
5. Resolve build definitions and produce application packages.
6. Execute application management operations.
7. Resolve target worker and application references.
8. Execute target operations.

Alias resolution is performed before the operation that consumes the corresponding alias. Resolved information may be retained internally so that subsequent operations can reuse it without repeating the same resolution or server query. For each target, target operations are executed according to the defined operation precedence rather than the order in which the operations appear in the source file.

The precedence is:

`unset` → `remove` → `deploy` → `push_only` → `set_active` → `disconnect`

Shell hooks associated with an operation are executed immediately before or after the corresponding operation, according to whether they are specified as `pre_deploy` or `post_deploy`.

3 File Syntax

3.1 HCL Syntax

Staploy file follows the syntax defined by HCL. A Staploy file is therefore composed of HCL blocks, labels, attributes, expressions, and values. This specification defines the semantic meaning of the HCL constructs used by Staploy file. Unless otherwise stated, lexical and syntactic rules are those of HCL.

A minimal Staploy file may contain a single top-level block:

```
configure {
   address = "127.0.0.1"
   port    = 18090
}
```

Multiple top-level blocks may be combined to describe a complete build and deployment workflow.

3.2 Blocks

A block is a named structural element containing a set of nested attributes and, where applicable, child blocks. Staploy file defines the following top-level block types:

- `configure` — defines global CLI and server configuration.
- `alias` — defines worker and application aliases.
- `build` — defines application package build operations.
- `manage` — defines application management operations.
- `target` — defines worker-targeted operations.

Some blocks may contain additional nested blocks. For example, a `target` may contain deployment operations:

```
target "production" {
   workers = ["group:all"]
   deploy "my-app" {
      post_deploy = "my-app --version"
   }
}
```

The set of valid child blocks and attributes is determined by the containing block and is specified in the corresponding sections of this document.

3.3 Labels

A label identifies a particular instance of a block. Labels are written immediately after the block type and are enclosed in quotation marks. For example:

```
build "my-app" {
   ...
}

target "production" {
   ...
}
```

The interpretation of a label depends on the block in which it occurs. For example, the label of a `target` identifies the target, while the label of an `alias` child block identifies the alias name. Some blocks may accept multiple labels when required by the HCL schema. The number and meaning of labels supported by each block are defined in the relevant section of this specification.

3.4 Attributes

An attribute assigns a value to a named property using the following form:

```
name = value
```

For example:

```
address      = "127.0.0.1"
port         = 18090
enforce_uuid = false
workers      = ["worker-1", "worker-2"]
```

The type and interpretation of an attribute are determined by the block in which it appears. Attributes may contain primitive values such as strings, numbers, and booleans, as well as collections such as lists. Some attributes may also accept expressions evaluated by the Staploy CLI.
An attribute may be optional, in which case its absence causes the implementation to use the default behavior defined for that attribute.

3.5 Expression

Expressions are values whose meaning is interpreted by Staployfile or evaluated during execution. Staployfile defines several expression forms in addition to ordinary HCL values. A notable example is the worker expression:

```
workers = [
"worker-1",
"group:all",
"alias:production"
]
```

The `group:` and `alias:` prefixes identify Staploy-specific expressions. These expressions are resolved according to the rules defined in the Worker Expressions and Resolution Rules sections. Application references may similarly use the `alias:` prefix:

```
deploy "alias:my-app" {
   ...
}
```

Shell expressions may also be used by attributes that explicitly support shell evaluation. Such expressions are evaluated by the CLI at the stage defined by the corresponding operation. An expression is not necessarily evaluated when the file is parsed. Evaluation and resolution occur at the stage required by the semantics of the containing block or operation.

4 Global Blocks

A Staployfile is organized into a set of top-level blocks. Each block represents a distinct stage or component of the Staploy workflow. The following top-level blocks are defined:

- `configure`
- `alias`
- `build`
- `manage`
- `target`

Each block has a specific purpose and may contain a defined set of attributes and nested blocks. Blocks that are not recognized by the Staployfile implementation are rejected during parsing.

4.1 `configure`

The `configure` block defines global configuration values used when processing the Staploy file. It may specify connection information for the Staploy server and global CLI behavior. Configuration defined in this block applies to operations within the current Staploy file unless overridden by an operation-specific mechanism.

A typical configuration is:

```
configure {
   address      = "192.168.50.194"
   port         = 18090
   enforce_uuid = false
}
```

The `configure` block is optional. When omitted, the CLI uses its default configuration and command-line configuration where applicable.

4.2 `alias`

The `alias` block defines named references that can be reused throughout a Staploy file.

Two alias types are currently supported:

- Worker aliases, which represent a set of workers.
- Application aliases, which represent an application name and, optionally, a version.

For example:

```
alias {
   worker "my-worker" {
      workers = ["group:all"]
   }

   app "my-app" {
      name    = "my-app"
      version = "1.2.3"
   }
}
```

Aliases are resolved when they are referenced by an operation. The detailed syntax and resolution rules for aliases are defined in Section 5.

4.3 `build`

The `build` block defines an application package build. A build describes the application name, output location, executable files, environment variables, architecture-specific build commands, and other metadata required to construct a Staploy package.

For example:

```
build "my-app" {
   version = "1.2.3"
   output_dir = "zig-out"
   executable = ["my-app"]

   x86_64 {
      pre_build = "./build-x86_64.sh"
      path      = "zig-out/x86_64/bin"
   }
}
```

A build may reference an application alias instead of specifying the application directly.

```
build "alias:my-app" {
   ...
}
```

Build processing produces a package that may subsequently be consumed by a `manage` or `target` block. The complete build syntax is defined in Section 8.

4.4 `manage`

The `manage` block defines operations that manage application packages independently of deployment to workers. Supported management operations include:

- `create`
- `upload`
- `delete`
- `pull`

For example:

```
manage "my-app" {
   upload {
      package_file = "my-app.tar"
   }
}
```

The application name may be specified directly or through an application alias. Management operations are executed after build processing and before target processing. Detailed management operation syntax is defined in Section 9.

4.5 `target`

The `target` block defines operations to be performed against a selected set of workers. A target has a label identifying the target and a `workers` attribute identifying the workers against which its operations are executed.

For example:

```
target "production" {
   workers = ["alias:production"]
   deploy "my-app" {
      post_deploy = "my-app --version"
   }
}
```

A target may contain one or more target operations. The operations are processed according to the target operation precedence defined by this specification rather than by their textual order within the block. The currently supported target operations are:

- `unset`
- `remove`
- `deploy`
- `push_only`
- `set_active`
- `disconnect`

Each target operation MAY occur at most once within a target block. A target MUST NOT contain multiple blocks of the same operation type. For example, a target containing two `deploy` blocks is invalid. Different

operation types MAY be specified together within the same target. This restriction applies independently to `unset`, `remove`, `deploy`, `push_only`, `set_active`, and `disconnect`.

Target operations may also define shell hooks that are executed before or after the corresponding operation. Detailed target operation syntax and execution semantics are defined in Section 10.

5 Alias

An alias provides a named reference to a set of resources or a specific application definition. Aliases allow other blocks to refer to resources without repeating their complete configuration. Aliases are declared in the global `alias` block and are resolved when a block references them using the `alias:` prefix.

5.1 Alias Model

The `alias` block defines named resources that can be referenced by other blocks in a Staploy file. An alias has a type and a name. The type determines the kind of resource represented by the alias.

```
alias {
   worker "my-workers" {
      ...
   }

   app "my-app" {
      ...
   }
}
```

The following alias types are currently supported:

- `worker`: Defines a reusable worker selection.
- `app`: Defines a reusable application reference.

An alias is referenced by prefixing its name with `alias:`.

```
build "alias:my-app" {
   ...
}

manage "alias:my-app" {
```

```
    ...
}

target "deploy" {
    workers = ["alias:my-workers"]
    ...
}
```

The `alias:` prefix explicitly indicates that the value is an alias reference rather than a literal resource name.

5.2 Worker Alias

A worker alias represents a set of workers selected from the workers currently known to the Staploy server. A worker alias is declared using the `worker` block.

```
alias {
    worker "zmx-deploy" {
        workers = ["group:all"]

        where {
            arch = ["x86_64", "arm", "aarch64", "mipsel"]
        }
    }
```

The `workers` attribute specifies the initial worker selection. Worker groups may be used to select workers by group. The optional `where` block further filters the selected workers according to their worker information. For example, the following alias selects workers belonging to the `all` group whose architecture is supported by the application:

```
worker "zmx-deploy" {
    workers = ["group:all"]

    where {
        arch = ["x86_64", "arm", "aarch64", "mipsel"]
    }
}
```

Worker aliases are resolved against the workers known to the server when the Staployfile is processed. Therefore, the result of a worker alias may differ

as workers join, leave, or change their reported properties. A worker alias can be used wherever a worker selection is accepted.

```
target "upload_zmx" {
   workers = ["alias:zmx-deploy"]
}
```

5.3 App Alias

An application alias provides a reusable reference to an application and its version information. An app alias is declared using the `app` block.

```
alias {
   app "zmx-build" {
      name = "zmx"
      version = "shell:git describe --tags --abbrev=0"
   }
}
```

The `name` attribute specifies the application name registered in the Staploy server. The `version` attribute specifies the version associated with the application. The value may be resolved dynamically using a supported value expression such as a shell command. An app alias can be referenced by build, management, and deployment blocks.

```
build "alias:zmx-build" {
   ...
}

manage "alias:zmx-build" {
   ...
}

target "upload_zmx" {
   deploy "alias:zmx-build" {
      ...
   }
}
```

Using an app alias allows the same application definition to be shared by multiple stages of a Staployfile without duplicating the application name or version definition. This is particularly useful when a build artifact is subsequently registered and deployed as part of the same workflow.

5.4 Alias Resolution

Alias references are resolved when the Staploy file is processed. An alias reference has the form of `alias:[ALIAS_NAME]`. The alias name is resolved within the aliases declared in the current Staploy file. The resolved alias must have a type compatible with the context in which it is referenced. For example, a worker alias may be used as a worker selection:

```
target "upload_zmx" {
   workers = ["alias:zmx-deploy"]
}
```

while an app alias may be used as an application reference:

```
build "alias:zmx-build" {
   ...
}
```

An alias reference does not itself create a new worker or application. It only provides an indirect reference to the corresponding alias definition. If an alias cannot be resolved, or if an alias of an incompatible type is used, processing of the Staploy file fails with an appropriate configuration error. Alias resolution also provides a separation between resource selection and resource execution. For example, a worker alias determines which workers participate in a target, while the target itself determines what operation is performed on those workers. This allows a single alias to be reused by multiple blocks while keeping resource selection independent of the execution workflow.

6 Worker Expressions

Worker expression specify the workers to which an operation is applied. A worker expression may identify a worker directly, select workers through a group, or refer to a previously defined worker alias. Worker expressions are used by blocks that operate on one or more workers, including target operations.

6.1 Worker ID

A worker ID identifies a worker by its unique identifier assigned by Staploy. A worker ID may be specified directly in a worker expression.

```
target "deploy" {
   workers = ["worker-8f3c2a"]
}
```

Worker IDs are expected to uniquely identify a worker. The worker ID is resolved against the workers known to the Staploy server. If a specified worker ID does not correspond to a known worker, the worker expression cannot be resolved.

6.2 Worker Name

A worker name identifies a worker using its configured or reported name. A worker name may be used where a worker expression accepts a direct worker reference.

```
target "deploy" {
   workers = ["zmx-worker-01"]
}
```

Unlike a worker ID, a worker name is not necessarily a globally unique identifier. If multiple workers match the specified name, the implementation MUST apply the worker resolution rules defined by the execution context. Worker names are therefore intended primarily for human-readable worker references. Worker IDs SHOULD be preferred when an unambiguous reference to a specific worker is required.

6.3 `group:` Expression

The `group:` expression selects workers belonging to a worker group. The general form is: `group:[GROUP_NAME]` For example:

```
target "deploy" {
   workers = ["group:production"]
}
```

A group expression represents a set of workers rather than a single worker. The set is determined from the workers known to the Staploy server and their associated group information. The special group name `all`, when supported by the implementation, represents all workers available to the current operation.

```
target "deploy" {
   workers = ["group:all"]
}
```

Group expressions may be combined with other worker expressions in the same worker selection.

```
target "deploy" {
   workers = [
   "worker-8f3c2a",
   "group:production"
   ]
}
```

The resulting worker set is resolved before the corresponding operation is executed.

6.4 `alias:` Expression

The `alias:` expression refers to a worker alias declared in the Staploy file. The general form is: `alias:[ALIAS_NAME]`.

For example:

```
alias {
   worker "production-workers" {
      workers = ["group:production"]
   }
}

target "deploy" {
   workers = ["alias:production-workers"]
}
```

A worker alias is resolved to the worker selection defined by the corresponding alias declaration. The `alias:` prefix distinguishes an alias reference from a direct worker ID, worker name, or group expression. An `alias:` expression MUST refer to a worker alias when used in a worker expression. Referencing an application alias or an undefined alias is a resolution error.

6.5 `where` Block

The `where` block filters a set of workers according to their reported attributes. A `where` block is applied to a worker selection obtained from a

worker expression. Each condition specifies a constraint on one worker attribute.

```
target "deploy" {
   workers = ["group:all"]

   where {
      arch = ["x86_64"]
      memory = "20G.."
      cpu = "!..10"
      name = "prefix:cuj1559-vm"
   }
}
```

All conditions specified in the same `where` block are combined conjunctively. A worker therefore remains in the resulting worker set only when it satisfies every specified condition.

6.5.1 Worker Attributes

The following worker attributes may be used as `where` conditions:

`name` The worker name.

`arch` The CPU architecture reported by the worker. Multiple values may be specified.

`cpu` The CPU information reported by the worker.

`memory` The memory information reported by the worker.

`workdir` The working directory reported by the worker.

`shell_enable` Whether shell execution is enabled for the worker.

`integrity_skip` Whether integrity checking is skipped for the worker.

Boolean attributes such as `shell_enable` and `integrity_skip` are specified as boolean values.

```
where {
   shell_enable = true
   integrity_skip = false
}
```

String and numeric attributes support filter expressions. The expression determines how the supplied value is compared with the corresponding worker attribute.

6.5.2 Exact Matching

The `exact:` prefix explicitly requests an exact string match.

```
where {
   name = "exact:cuj1559-vm-01"
}
```

The condition is satisfied only when the complete attribute value is equal to the specified value. The `exact:` prefix is useful when a more general string matching rule could otherwise be applicable. If no filter prefix is specified for a string attribute, the condition is interpreted as an exact match.

6.5.3 Prefix Matching

The `prefix:` prefix matches strings beginning with the specified value.

```
where {
   name = "prefix:cuj1559-vm"
}
```

The condition above matches worker names such as `cuj1559-vm-01` and `cuj1559-vm-02`, provided that the complete worker name begins with `cuj1559-vm`.

6.5.4 Suffix Matching

The `suffix:` prefix matches strings ending with the specified value.

```
where {
   name = "suffix:-prod"
}
```

The condition matches worker names whose complete value ends with `-prod`.

6.5.5 Regular Expression Matching

The `regex:` prefix interprets the remainder of the expression as a regular expression.

```
where {
   name = "regex:^cuj[0-9]+-vm-[0-9]+$"
}
```

A worker satisfies the condition when the corresponding attribute matches the specified regular expression. The regular expression syntax and behavior are determined by the regular expression implementation used by Staploy.

6.5.6 Numeric Range Matching

Numeric attributes support range expressions using the `..` operator. A range may specify a lower bound, an upper bound, or both.

```
where {
   memory = "20G.."
}
```

The expression above specifies a lower-bounded range and matches values greater than or equal to `20G`. An upper-bounded range may be specified by placing `..` before the value.

```
where {
   memory = "..64G"
}
```

A bounded range may specify both endpoints.

```
where {
   memory = "4G..64G"
}
```

The range operator is primarily intended for numeric worker attributes such as memory and CPU-related values.

6.5.7 Negation

The `!` prefix negates a filter expression.

```
where {
   name = "!prefix:test-"
}
```

The condition above matches worker names that do not satisfy the `prefix:test` expression. Negation may be combined with other filter expressions.

For example:

```
where {
   cpu = "!..10"
}
```

This applies the negated form of the corresponding range expression.

6.5.8 Filter Selection

The filter operators are summarized below.

Expression	Meaning
`exact:VALUE`	Exact string match
`prefix:VALUE`	Prefix string match
`suffix:VALUE`	Suffix string match
`regex:EXPR`	Regular expression match
`VALUE..`	Lower-bounded numeric range
`..VALUE`	Upper-bounded numeric range
`VALUE..VALUE`	Bounded numeric range
`!EXPR`	Negation of a filter expression

These expressions may be used wherever the corresponding worker attribute supports the relevant comparison operation. An implementation MUST reject a filter expression that is not applicable to the type of the selected worker attribute. For example, a numeric range expression MUST NOT be interpreted as a string prefix or suffix expression merely because the attribute is represented as a string in the Staploy file.

6.5.9 Combined Conditions

Multiple conditions may be combined to construct a more specific worker selection.

```
target "filter_test" {
   workers = ["group:all"]

   where {
      arch = ["x86_64"]
      memory = "20G.."
      cpu = "!..10"
      name = "prefix:cuj1559-vm"
   }

   set_active "staploy-worker" {
      version = "1.0.0"
```

```
    }
}
```

The example selects workers satisfying all the following conditions:

- the worker architecture is `x86_64`;
- the worker has at least `20G` of memory;
- the worker's CPU value does not satisfy the specified upper-bounded range;
- the worker name begins with `cuj1559-vm`.

The `where` block does not itself select the initial worker population. Instead, it filters the workers produced by the preceding worker expression. If no worker satisfies the combined conditions, the resulting worker set is empty. The handling of an empty worker set is determined by the operation using the worker selection.

If worker resolution or filtering produces an empty worker set, the target is skipped and reported as having no matching workers. The target MUST NOT be reported as successfully executed.

7 Application References

Application reference identifies an application and, when required, a specific version to be used by an operation. An application reference may identify an application directly, refer to an application alias, or obtain a version from a shell command.

7.1 Application Name

An application name identifies an application registered on the Staploy server. It is specified as the application identifier in an operation that accepts an application reference.

```
set_active "staploy-worker" {
    version = "1.0.0"
}
```

The application name is resolved against the applications known to the Staploy server. If no application with the specified name exists, application resolution fails.

7.2 Application Version

An application version identifies a specific version of an application.

```
set_active "staploy-worker" {
   version = "1.0.0"
}
```

The version value is interpreted as an application version identifier. The specified version MUST be available for the referenced application when the operation requires an existing application artifact. Version selection is independent of worker selection. If an operation targets multiple workers with different architectures, the selected application version MUST provide an artifact compatible with each worker to which the operation is applied.

7.3 `alias:` Expression

The `alias:` expression refers to an application alias declared in the Staployfile. For example:

```
alias {
   app "latest-worker" {
      name = "staploy-worker"
      version = "1.0.0"
   }
}
```

An application alias is resolved to the application reference represented by the corresponding alias declaration. The `alias:` prefix explicitly identifies the reference as an alias. A worker alias MUST NOT be used where an application reference is required, and an undefined application alias results in a resolution error.

7.4 `shell:` Expression

The `shell:` expression obtains an application reference value by executing a shell command.

The general form is:

```
shell:
```

For example:

```
alias {
    app "staploy-worker" {
        version = "shell:git describe --tags --abbrev=0"
    }
}
```

The shell command is executed by the Staploy client and its resulting output is used as the value of the corresponding application reference. This permits application versions to be determined dynamically rather than being hard-coded in the Staployfile. For example, a project may derive the version from a Git tag or another external versioning mechanism. The command MUST complete successfully for the expression to resolve successfully. The resulting value is interpreted according to the context in which the `shell:` expression occurs. Shell expressions are evaluated during execution of the corresponding operation. Consequently, their result may differ between executions of the same Staployfile. The use of `shell:` expressions may depend on the shell execution environment of the Staploy client. Implementations SHOULD document the shell, environment variables, working directory, and output handling used when evaluating shell expressions.

8 Build

The `build` block defines the procedure for producing an application package. A build specifies the application name, output directory, executable files, build environment, optional version information, and one or more target configurations. A build may contain architecture-specific targets as well as files shared by all targets. Build commands may be specified globally or for an individual target.

8.1 Build Block

A build block is identified by an application name and has the following general form:

```
build "my-app" {
   output_dir = "./build"
   executable = ["my-app"]

   x86_64 {
      path = "./bin/amd64"
   }
}
```

The label of the build block specifies the application name to which the resulting artifacts belong. A build block may contain the following attributes and target blocks:

- `output_dir`
- `version`
- `executable`
- `envs`
- `lib_version`
- `pre_build`
- `post_build`
- `share`
- architecture-specific target blocks

The `output_dir` and `executable` attributes define the basic output of a build. Target blocks determine which files are associated with each target architecture.

8.2 Build Attributes

8.2.1 `output_dir`

The `output_dir` attribute specifies the directory used as the output of the build.

```
build "my-app" {
   output_dir = "./build"
   ...
}
```

The output directory is used as the base location from which the resulting application files are collected.

8.2.2 `version`

The optional `version` attribute specifies the version associated with the resulting application.

```
build "my-app" {
   output_dir = "./build"
   version = "1.2.0"
   ...
}
```

If omitted, the version may be determined by the surrounding build or application resolution semantics.

8.2.3 `executable`

The `executable` attribute specifies the names of executable files contained in the resulting application.

The values are file names rather than filesystem paths.

```
build "my-app" {
   output_dir = "./build"

   executable = [
   "my-app",
   "helper"
   ]
   ...
}
```

The specified names are registered as executable files in the resulting application package. They identify files in the build output and are not interpreted as source paths.

8.2.4 `envs`

The optional `envs` attribute specifies environment variables used while executing the build.

```
build "my-app" {
   output_dir = "./build"

   envs = [
   "CGO_ENABLED=0",
   "GOFLAGS=-trimpath"
   ]

   ...
```

```
}
```

The supplied values are passed to the build environment according to the shell execution semantics of the implementation.

8.2.5 `lib_version`

The optional `lib_version` attribute specifies the library version associated with the build.

```
build "my-app" {
   output_dir = "./build"
   lib_version = "1.4.0"
   ...
}
```

The interpretation of this value is dependent on the application and its build process.

8.3 Target Architectures

Architecture-specific build targets are represented by nested blocks. Each target identifies the files and optional commands associated with one architecture. The following architecture-specific target blocks are supported:

Block	Architecture
`i386`	32-bit x86
`x86_64`	64-bit x86
`arm`	32-bit ARM
`aarch64`	64-bit ARM
`riscv32`	32-bit RISC-V
`riscv64`	64-bit RISC-V
`mipsel`	Little-endian MIPS
`mips64el`	64-bit little-endian MIPS

A target block has the following form:

```
x86_64 {
   path = "./bin/amd64"
}
```

The `path` attribute identifies the path containing the files associated with the target. Different target blocks may therefore refer to different build outputs.

```
build "my-app" {
   output_dir = "./build"

   x86_64 {
      path = "./bin/amd64"
   }

   aarch64 {
      path = "./bin/arm64"
   }
}
```

The resulting application package for each architecture is constructed from the corresponding architecture-specific target together with the files specified by the `share` block.

8.3.1 `share` Target

The `share` block specifies files common to all target architectures.

Unlike an architecture-specific target, `share` does not represent a CPU architecture. Its contents are included in the output of every architecture-specific target.

```
build "my-app" {
   output_dir = "./build"

   share {
      path = "./resources"
   }

   x86_64 {
      path = "./bin/amd64"
   }

   aarch64 {
      path = "./bin/arm64"
   }
}
```

In this example, the files contained in `./resources` are shared by both the `x86_64` and `aarch64` outputs. A shared file is therefore not required to be duplicated in each architecture-specific target.

8.4 Pre-build Commands

The `pre_build` attribute specifies a command to be executed before the corresponding build stage. A build-level `pre_build` command applies to the build as a whole.

```
build "my-app" {
   output_dir = "./build"
   pre_build = "go generate ./..."

   ...
}
```

A target may additionally define its own `pre_build` command.

```
build "my-app" {
   output_dir = "./build"

   x86_64 {
      path = "./bin/amd64"
      pre_build = "make build-amd64"
   }
}
```

A target-level command applies only to the corresponding target, while a build-level command applies to the build as a whole. The order in which build-level and target-level commands are executed is defined by the execution semantics of the build processor.

8.5 Build Output

The build output consists of the files collected from the selected target and the corresponding `share` target. For an architecture-specific target, the target's `path` identifies the target-specific files.

```
build "my-app" {
   output_dir = "./build"

   executable = [
   "my-app"
   ]

   share {
```

```
      path = "./resources"
   }

   x86_64 {
      path = "./bin/amd64"
   }
}
```

The resulting output contains both the architecture-specific files from `./bin/amd64` and the shared files from `./resources`.

The `executable` attribute does not specify the source location of these files. It specifies which resulting files are considered executable files. If the expected build output cannot be located or processed, the build operation fails. The resulting artifacts are subsequently available to operations such as upload and deployment through build artifact resolution.

8.6 Build Alias Resolution

A build may be referenced indirectly through an application alias. When an application reference resolves to an application associated with a build definition, the corresponding build definition may be used to produce the application artifact. Build alias resolution therefore connects an application reference with the build definition that produces its artifact.

For example:

```
alias {
   app "my-app" {
      ...
   }
}

build "my-app" {
   output_dir = "./build"

   executable = [
   "my-app"
   ]

   x86_64 {
      path = "./bin/amd64"
   }
}
```

The alias does not modify the build definition. It provides an alternative name through which the application or build can be referenced. If an alias cannot be resolved to a valid application or build definition, the corresponding operation fails during resolution. Build alias resolution precedes build artifact resolution. The former identifies the application and its associated build definition, while the latter identifies the concrete artifact corresponding to the target architecture.

9 Manage Operations

The `manage` block defines operations for managing applications and application packages in the local Staploy server. A manage block is associated with an application by its label:

```
manage "my-app" {
   ...
}
```

A `manage` block may contain one or more of the following operations:

- `create` — create an application in the local Staploy server.
- `upload` — upload a local application package to the local Staploy server.
- `delete` — delete application versions from the local Staploy server.
- `pull` — import an application package from an external Staploy repository into the local Staploy server.

Each operation is independent and is executed only when explicitly specified.

9.1 Manage Block

The general form of a `manage` block is:

```
manage "my-app" {
   create {
      ...
   }

   upload {
      ...
   }
```

```
    delete {
        ...
    }

    pull "my-app" {
        ...
    }
}
```

The label of the `manage` block identifies the application to which the management operations apply. The operations may be combined in a single block. They are processed according to the execution semantics defined in Section 12.

9.2 `create`

The `create` operation creates an application in the local Staploy server. It accepts an optional `description` attribute:

```
manage "my-app" {
    create {
        description = "My application"
    }
}
```

The `description` attribute specifies the description associated with the application. If omitted, the application is created without an explicitly specified description. The application is created in the local Staploy server to which the operation is submitted.

9.3 `upload`

The `upload` operation uploads an application package from the local filesystem to the local Staploy server. It accepts an optional `path` attribute specifying the package file:

```
manage "my-app" {
    upload {
        path = "./build/my-app.tar"
    }
}
```

The `path` attribute identifies the local package file to upload. If `path` is omitted, the package is resolved according to the applicable build artifact resolution rules. The package is transferred to the local Staploy server and registered as a version of the application identified by the enclosing `manage` block. The operation does not upload the package to an external repository. The destination is always the local Staploy server associated with the current execution.

9.4 `delete`

The `delete` operation deletes application versions from the local Staploy server. Specific versions may be specified using the `versions` attribute:

```
manage "my-app" {
   delete {
      versions = [
      "1.0.0",
      "1.1.0"
      ]
   }
}
```

Each value identifies an application version to be deleted.

If the `versions` attribute is omitted, all versions of the application are deleted.

```
manage "my-app" {
   delete {}
}
```

An empty `delete` block therefore represents an explicit request to delete all versions of the application. The operation applies to the application identified by the label of the enclosing `manage` block.

9.5 `pull`

The `pull` operation imports an application package from an external Staploy repository into the local Staploy server. The application to import is identified by the label of the `pull` block:

```
manage "my-app" {
   pull "my-app" {
      version = "1.2.0"
```

```
        repository = "https://repository.example.com"
    }
}
```

The `name` label identifies the application to retrieve from the external repository. The optional `version` attribute specifies the application version to import. If omitted, the version is resolved according to the application resolution rules. The `repository` attribute specifies the URL of the external Staploy repository. The operation retrieves the requested application package from the external repository and registers it in the local Staploy server.

For example:

```
manage "my-app" {
    pull "my-app" {
        repository = "https://repository.example.com"
    }
}
```

In contrast to `upload`, which transfers a package from the local filesystem to the local server, `pull` transfers an application package from an external Staploy repository to the local server. The `pull` operation therefore does not merely download a package to the local filesystem. The retrieved application and package are registered in the local Staploy server and can subsequently be managed or deployed using the local repository. The specified `repository` MUST identify an accessible Staploy repository endpoint. If the requested application or version cannot be resolved in the external repository, or if the package cannot be retrieved or registered in the local server, the `pull` operation fails.

10 Target Operations

The `target` block defines operations to be performed on one or more workers. A target consists of a worker selection and one or more operations. The selected workers are determined by the `workers` attribute and, optionally, a `where` block.

10.1 Target Block

A `target` block is identified by a label and contains the workers on which its operations are performed.

```
target "production" {
   workers = ["group:production"]

   deploy "my-app" {
      version = "1.2.0"
   }
}
```

The `workers` attribute specifies the initial set of workers targeted by the block. A target may contain multiple operations. Each operation is applied to the workers selected by the target.

10.2 Worker Selection

Workers are selected using the `workers` attribute:

```
target "production" {
   workers = [
   "worker-01",
   "worker-02",
   "worker-03"
   ]

   ...
}
```

Worker expressions may refer to worker IDs, worker names, groups, or aliases. Multiple expressions may be specified simultaneously. A `where` block may be used to further restrict the selected workers:

```
target "production" {
   workers = ["group:all"]

   where {
      arch   = ["x86_64"]
      memory = "20G.."
      cpu    = "!..10"
      name   = "prefix:production-"
   }

   ...
}
```

The `where` conditions are applied to the workers selected by `workers`. A worker must satisfy the specified conditions to remain in the target set. If multiple conditions are specified, all conditions must be satisfied. The syntax and semantics of worker expressions and `where` conditions are defined in Section 6.

10.3 `unset`

The `unset` operation removes the currently active version of an application from a worker.

```
target "maintenance" {
   workers = ["group:production"]

   unset "my-app" {
   }
}
```

The `unset` operation is equivalent to a `set_active` operation with an empty version. It therefore clears the active version of the specified application without removing the application package from the worker. The optional `pre_deploy` and `post_deploy` attributes may be used to execute shell hooks before and after the operation.

10.4 `remove`

The `remove` operation removes application versions from the selected workers.

A specific version may be removed by specifying the `version` attribute:

```
target "cleanup" {
   workers = ["group:all"]

   remove "my-app" {
      version = "1.0.0"
   }
}
```

If `version` is omitted, all versions of the specified application are removed from the worker:

```
target "cleanup" {
   workers = ["group:all"]
```

```
   remove "my-app" {
   }
}
```

The optional `autoremove` attribute provides a less destructive cleanup operation. When enabled, all versions that are not currently active are removed, while the currently active version is retained.

```
target "cleanup" {
   workers = ["group:all"]

   remove "my-app" {
      autoremove = true
   }
}
```

When `autoremove` is enabled, the `version` attribute is not required. The operation removes every inactive version of the application while preserving its currently active version. The optional `pre_deploy` and `post_deploy` attributes may be used to execute shell hooks before and after the operation.

10.5 `deploy`

The `deploy` operation pushes an application version to the selected workers and activates the resulting version.

```
target "production" {
   workers = ["group:production"]

   deploy "my-app" {
      version = "1.2.0"
   }
}
```

The `name` label identifies the application to deploy.

The optional `version` attribute specifies the version to deploy. If omitted, the applicable application resolution rules are used to determine the version. Unlike `push_only`, a successful deployment results in the specified version becoming active on the worker. The optional `pre_deploy` and `post_deploy` attributes specify shell commands to execute before and after the deployment.

10.6 push_only

The push_only operation pushes an application version to the selected workers without activating it.

```
target "staging" {
   workers = ["group:staging"]

   push_only "my-app" {
      version = "1.2.0"
   }
}
```

The name label identifies the application to push.

The optional version attribute specifies the version to push. If omitted, the applicable application resolution rules are used to determine the version. After a successful operation, the pushed version is available on the worker but is not selected as the active version. This operation can therefore be used to distribute an application version before activating it. The optional pre_deploy and post_deploy attributes specify shell commands to execute before and after the operation.

10.7 set_active

The set_active operation changes the active version of an application on the selected workers.

```
target "production" {
   workers = ["group:production"]

   set_active "my-app" {
      version = "1.2.0"
   }
}
```

The name label identifies the application. The version attribute specifies the version to activate.

The specified version must already be available on the worker. The operation does not push a package to the worker. An empty version has the same effect as the unset operation and clears the active version. The optional pre_deploy and post_deploy attributes specify shell commands to execute before and after the operation.

10.8 `disconnect`

The `disconnect` operation disconnects the selected workers from the Staploy server.

```
target "maintenance" {
   workers = ["group:maintenance"]

   disconnect {
   }
}
```

Unlike application operations, `disconnect` does not require an application name or version. The operation terminates the workers' current connection to the Staploy server. It does not remove applications or modify the application state stored on the worker. The optional `pre_deploy` attribute specifies a shell command to execute before disconnecting the worker.

11 Shell Hooks

Shell hook allow shell commands to be executed as part of target operations. Shell hooks are distinct from the `shell:` expression. The `shell:` expression is used to obtain a value by executing a command and is available only in `app` and `build` contexts. Shell hooks, in contrast, are associated with target operations and are executed on the target worker as part of the operation lifecycle.

11.1 Hook Model

Target operations may define commands to be executed before and after the operation. The available hooks depend on the operation:

Operation	`pre_deploy`	`post_deploy`
`deploy`	Available	Available
`push_only`	Available	Available
`set_active`	Available	Available
`unset`	Available	Available
`remove`	Available	Available
`disconnect`	Available	Not available

A hook is executed in the context of the worker on which the corresponding target operation is performed.

For example:

```
target "production" {
   workers = ["group:production"]

   deploy "my-app" {
      version = "1.2.0"

      pre_deploy  = "systemctl stop my-app"
      post_deploy = "systemctl start my-app"
   }
}
```

Hooks are optional. If a hook is not specified, no shell command is executed for that phase.

11.2 pre_deploy

The `pre_deploy` attribute specifies a shell command to execute before the associated target operation.

```
deploy "my-app" {
   version = "1.2.0"
   pre_deploy = "systemctl stop my-app"
}
```

The command is executed on the target worker before the operation itself is performed. For operations that transfer or modify an application, the hook therefore runs before the corresponding application operation. For `disconnect`, the hook is executed before the worker is disconnected from the Staploy server. If the `pre_deploy` command fails, the associated operation is considered to have failed and the operation is not executed.

11.3 post_deploy

The `post_deploy` attribute specifies a shell command to execute after the associated target operation.

```
deploy "my-app" {
   version = "1.2.0"
   post_deploy = "systemctl restart my-app"
}
```

The command is executed on the target worker after the associated operation has completed successfully. If the operation itself fails, the `post_deploy` hook is not executed. A failure of the `post_deploy` command causes the target operation to be reported as failed, even though the underlying application operation may already have completed.

11.4 Shell Execution

Shell hooks are executed by the Staploy worker on the target worker. The command is interpreted by the worker's shell environment. Consequently, the command syntax and available programs depend on the operating system and environment of the target worker. Shell commands are not evaluated by the Staploy server. The execution environment is the worker environment in which the associated target operation is performed. Commands may therefore access files, environment variables, and programs available on that worker. For example, a deployment may stop an existing service before deployment and restart it after the deployment:

```
target "production" {
   workers = ["group:production"]

   deploy "my-app" {
      version = "1.2.0"

      pre_deploy  = "systemctl stop my-app"
      post_deploy = "systemctl start my-app"
   }
}
```

In this example, both commands are executed by the worker. The Staploy server does not execute either command itself. The `shell:` expression is a separate mechanism. The shell: expression is used to obtain a value by executing a command. It is supported only in attributes explicitly defined as shell-evaluated, such as application alias version attributes, build version attributes, and build `lib_version` attributes. For example, an application alias may use the output of a shell command as a version:

```
alias {
   app "my-app" {
      version = "shell:git describe --tags --abbrev=0"
   }
}
```

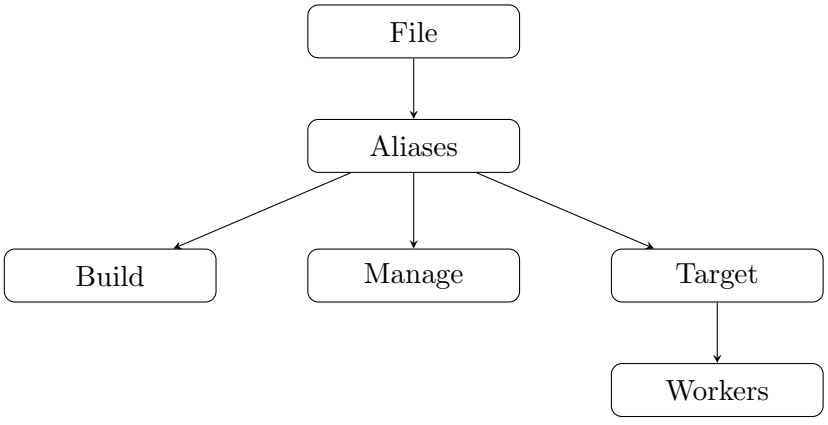
Similarly, a build may use a shell command to determine a build value:

```
build "my-app" {
   output_dir = "./build"
   version = "shell:git describe --tags --abbrev=0"

   executable = [
   "my-app"
   ]

   x86_64 {
      path = "./bin/amd64"
   }
}
```

The `shell:` expression is evaluated while resolving the corresponding configuration value. It is therefore distinct from shell hooks, which are executed as part of a target operation on the target worker. The `shell:` expression MUST NOT be used as a general-purpose shell hook in target operations.

12 Execution Semantics

The execution semantics define how a Staploy file is processed after it has been parsed. Execution consists of several stages. The implementation first reads the configuration file and resolves the objects referenced by each block. It then performs build, manage, and target operations according to their respective semantics. The processing of a Staploy file can be summarized as:



The exact order of independent operations is implementation-dependent unless an ordering is explicitly required by their semantics.

12.1 File Processing

File process parses the Staploy file and constructs its internal representation. The configuration file MUST be syntactically valid before any operation is executed. Syntax errors prevent further processing of the file. During this stage, blocks, attributes, labels, and nested blocks are identified according to the Staploy file syntax. The resulting configuration is then used as the input to alias, build, manage, and target processing. File processing does not itself execute build commands, shell hooks, or worker operations.

12.2 Alias Processing

Alias process resolves aliases declared in the Staploy file. Aliases provide alternative references to workers and applications and may therefore be used by subsequent expressions. Worker aliases are resolved when evaluating worker expressions. Application aliases are resolved when evaluating application references. An alias MUST resolve to a valid object of the corresponding type before it can be used. Alias resolution may involve additional expressions, including `where` conditions and `shell:` expressions where permitted. The result of alias processing is not itself an operation. It provides resolved references for subsequent processing stages.

12.3 Build Processing

Build process produces application artifacts from `build` blocks. A build is first resolved into its application, version, environment, target architectures, and output paths. Build-level commands and target-specific commands are then executed as required by the build definition. The resulting files are collected from the selected architecture target and the corresponding `share` target. The files specified by the `executable` attribute identify executable files in the resulting artifact. The values of this attribute are file names, not source paths. A build may produce artifacts for multiple target architectures. For example:

```
build "my-app" {
   output_dir = "./build"

   executable = [
   "my-app"
   ]

   share {
      path = "./resources"
   }
```

```
    x86_64 {
       path = "./bin/amd64"
    }

    aarch64 {
       path = "./bin/arm64"
    }
}
```

The resulting artifacts are made available to subsequent operations such as `upload`, `push_only`, and `deploy`. If a required build input, target, or artifact cannot be resolved, build processing fails and the affected operation is not executed.

12.4 Manage Processing

Manage process executes operations defined by `manage` blocks. The application identified by the `manage` block is resolved before the corresponding operation is performed. The operations modify or retrieve application data associated with the local Staploy server:

- `create` creates an application.
- `upload` transfers a local application package to the local server.
- `delete` removes application versions from the local server.
- `pull` retrieves an application package from an external Staploy repository and registers it in the local server.

For operations that require an application package, the required application and version are resolved before execution. Manage processing does not directly perform worker operations. Its scope is the application repository maintained by the Staploy server.

12.5 Target Processing

Target process executes operations defined by `target` blocks against selected workers. Worker selection is performed before the target operations are executed. The initial worker set is obtained from the `workers` attribute. If a `where` block is present, its conditions are applied to the selected workers and only workers satisfying all conditions remain in the target set. For each selected worker, the application and version required by the operation are resolved before the operation is submitted to the worker. Target operations are then executed according to the operation specified in the target block:

- `deploy` pushes and activates an application version.
- `push_only` pushes an application version without activating it.
- `set_active` activates an already available application version.
- `unset` clears the active application version.
- `remove` removes application versions from the worker.
- `disconnect` disconnects the worker from the server.

Shell hooks associated with a target operation are executed as part of target processing. A failure affecting one worker does not inherently imply that operations for other selected workers are canceled. The handling of individual and partial failures is defined by the error semantics.

13 Resolution Semantics

Resolution is the process of converting references and expressions in a Staployfile into concrete objects used during execution. Resolution is performed separately for workers, applications, aliases, and build artifacts. The result of each resolution MUST satisfy the constraints applicable to the corresponding object. Resolution does not itself perform an operation on a worker or modify the local Staploy server. It produces the concrete values required by subsequent execution stages.

13.1 Worker Resolution

Worker resolution converts worker expressions into a set of concrete workers. A worker expression may identify a worker directly by its ID or name, or indirectly through a group or worker alias.

For example:

```
target "production" {
   workers = [
   "worker-01",
   "worker-02",
   "group:production",
   "alias:production-workers"
   ]

   ...
}
```

The following forms are supported:

- A worker ID resolves to the corresponding worker.
- A worker name resolves to the corresponding worker.
- A `group:` expression resolves to all workers belonging to the specified group.
- An `alias:` expression resolves according to the corresponding worker alias.

If multiple expressions resolve to the same worker, the worker MUST occur only once in the resulting worker set. If an expression cannot be resolved, the target resolution fails. After the initial worker expressions have been resolved, a `where` block, when present, is applied to the resulting worker set.

13.2 Application Resolution

Application resolution converts an application reference into a concrete application and, when required, a concrete application version. An application may be referenced directly by its name or indirectly through an application alias. For operations that require a version, the version may be explicitly specified:

```
deploy "my-app" {
   version = "1.2.0"
}
```

When the version is omitted, Staploy resolves an applicable version according to the context of the operation. For target operations, version resolution may depend on the selected worker. This is necessary when the application contains multiple architecture-specific versions or artifacts. An application reference MUST resolve to an application available to the operation being performed. A requested version MUST also be available for the selected worker when the operation requires an architecture-compatible artifact.

13.3 Alias Resolution

Alias resolution converts an alias reference into the object or expression represented by the alias. Worker aliases and application aliases are resolved independently according to their respective definitions. An alias may contain expressions that require further resolution. Alias resolution therefore proceeds recursively until the referenced object can be determined.

For example:

```
alias {
   worker "production" {
      ...
   }

   app "release" {
      ...
   }
}

target "deploy-production" {
   workers = ["alias:production"]

   deploy "alias:release" {
      ...
   }
}
```

An alias MUST resolve to an object of the expected type. A worker alias cannot be used where an application reference is required, and an application alias cannot be used as a worker expression. Alias resolution MUST terminate. Circular aliases or other references that prevent resolution from reaching a concrete object are invalid. Where an alias contains a `where` block, its conditions are evaluated against the corresponding application or worker objects according to the alias semantics.

13.4 Build Artifact Resolution

Build artifact resolution determines the package or artifact to be used by an operation from the results of a `build` block. A build may produce artifacts for multiple target architectures. The applicable artifact is selected according to the architecture of the target worker when the operation is worker-specific. For example, a build may contain:

```
build "my-app" {
   output_dir = "./build"

   executable = [
   "my-app"
   ]

   share {
      path = "./resources"
```

```
    }

    x86_64 {
       path = "./bin/x86_64"
    }

    aarch64 {
       path = "./bin/aarch64"
    }
}
```

The `share` target contains files common to all architectures. Architecture-specific targets provide files applicable to their respective architectures. When resolving an artifact for a worker, Staploy selects the target corresponding to the worker's architecture and combines it with the common files from the `share` target. The names specified by the `executable` attribute identify executable files in the resulting artifact. They are file names and do not identify source paths. If no compatible build artifact exists for the selected worker, artifact resolution fails.

13.5 Resolution Constraints

Resolution MUST produce an unambiguous and valid result before the corresponding operation is executed. The following constraints apply:

- A worker reference MUST resolve to an existing worker.
- A worker alias MUST resolve to a worker reference or worker set.
- An application reference MUST resolve to an existing application when the operation requires an existing application.
- An application alias MUST resolve to an application reference.
- A requested application version MUST exist when an existing version is required.
- A worker-specific application artifact MUST be compatible with the architecture of the selected worker.
- An alias MUST NOT form a circular resolution chain.
- A resolution MUST NOT silently produce an ambiguous result.

Resolution failures prevent the affected operation from being executed. For target operations involving multiple workers, resolution is performed

in the context of each selected worker when the result depends on worker-specific properties such as architecture. Consequently, the same application reference may resolve to different concrete artifacts for different workers while still representing the same logical application and version.

14 Error Semantics

Staploy distinguishes errors according to the stage at which they occur. An error in one operation or worker does not necessarily terminate execution of the remaining operations. Unless otherwise specified, processing continues with the next applicable operation or worker after an error has been reported.

14.1 Syntax Errors

A syntax error occurs when a Staployfile cannot be parsed according to the Staployfile syntax. Examples include malformed blocks, invalid attribute syntax, invalid labels, and values that cannot be decoded into the expected configuration type. A syntax error prevents the affected Staployfile from being processed. No operation depending on a successfully parsed configuration is executed when parsing fails.

14.2 Resolution Errors

A resolution error occurs when a reference cannot be resolved to the object required by an operation. Examples include an unknown worker, unknown application, unavailable application version, or unresolved alias. For target processing, a failure to resolve worker expressions causes the affected target to be skipped. Similarly, failure to resolve an application or application version prevents the corresponding operation from being executed. Resolution errors are reported to the user and do not necessarily terminate processing of subsequent targets or operations.

14.3 Build Errors

A build error occurs when a requested build cannot produce a valid application artifact. Build errors may result from invalid build configuration, failure of a build command, missing build output, unsupported target architecture, or failure while packaging the resulting artifact. A failed build MUST NOT be used as a successfully produced artifact by a subsequent operation. The affected build operation is reported as failed. Other independent operations may continue unless execution of those operations depends on the failed artifact.

14.4 Execution Errors

An execution error occurs while performing an operation after the required references have been resolved. Execution errors may occur during communication with a worker, application deployment, application activation, application removal, package transfer, or shell hook execution. For operations performed on multiple workers, an execution error affecting one worker does not terminate execution for the remaining workers. For example, if a deployment is requested for three workers and deployment fails on the first worker, the remaining workers are still processed. Shell hook failures have additional semantics.

A failure of a `pre_deploy` hook aborts the associated operation. The operation itself is not executed after the hook failure. A failure of a `post_deploy` hook does not undo an operation that has already completed. The hook failure is reported separately.

14.5 Partial Failure

A multi-worker operation may produce different results for different workers. Such a result is considered a partial failure when at least one worker succeeds and at least one worker fails.

For example:

```
target "production" {
   workers = [
   "worker-01",
   "worker-02",
   "worker-03"
   ]

   deploy "my-app" {
      version = "1.2.0"
   }
}
```

If the operation succeeds on `worker-01` and `worker-02` but fails on `worker-03`, the operation has partially failed. A failed worker does not cause successful operations on other workers to be rolled back. Staploy does not provide transactional rollback across workers for target operations. Consequently, after a partial failure, different workers may be left in different states. The CLI reports failures for individual workers while continuing execution where possible. A partial failure therefore indicates that the requested operation was not uniformly completed across all selected workers; it does not imply that the operation failed for every worker.

15 Conformance

A Staploy implementation conforms to this specification if it accepts valid Staployfiles according to the syntax and semantics defined in this document and applies the corresponding resolution and execution semantics. An implementation MAY provide additional features, attributes, or expressions, provided that they do not change the meaning of constructs defined by this specification.

15.1 Valid Staployfiles

A Staployfile is valid when all of the following conditions are satisfied:

- The file conforms to the Staployfile syntax.
- All required attributes and labels are present.
- All attribute values can be decoded into their declared types.
- All references used by an operation can be resolved when the operation is executed.
- Alias references resolve to objects of the expected type.
- Worker expressions resolve to valid workers or worker sets.
- Application references resolve to valid applications where required.
- Application versions and build artifacts satisfy the constraints applicable to the operation.
- Build blocks contain sufficient information to produce the requested artifact.

A syntactically valid file may nevertheless fail during resolution, build, or execution. Validity of the Staployfile therefore does not imply that every operation contained in the file will succeed. For example, the following file may be syntactically valid even when the referenced worker is not currently available:

```
target "production" {
   workers = ["worker-01"]

   deploy "my-app" {
      version = "1.2.0"
   }
}
```

The failure to resolve or communicate with `worker-01` is an execution or resolution error rather than a syntax error.

15.2 Implementation Requirements

A conforming implementation MUST:

- parse the Staployfile according to the syntax defined in this specification;
- distinguish worker references from application references according to their respective contexts;
- support worker and application alias resolution;
- evaluate `where` conditions according to the defined filter semantics;
- resolve build artifacts according to the target worker architecture where applicable;
- execute manage and target operations according to their defined semantics;
- execute shell hooks in the worker context;
- distinguish `pre_deploy` and `post_deploy` hook failures according to their respective semantics;
- report errors without incorrectly treating partial success as complete success;
- preserve successful operations when another worker subsequently fails, unless an operation explicitly defines rollback semantics.

A conforming implementation MUST NOT silently reinterpret a construct defined by this specification. In particular, an implementation MUST NOT treat an unresolved reference as a successfully resolved reference, execute an operation after a failed `pre_deploy` hook, or report a partially successful multi-worker operation as uniformly successful. Implementations MAY differ in their user interface, logging format, concurrency model, transport implementation, and internal representation, provided that the observable behavior required by this specification remains equivalent.

16 Examples

This section provides examples of common Staployfile configurations. The examples are intended to demonstrate how the constructs defined in this specification can be combined into practical workflows.

16.1 Minimal Staploy file

The following is a minimal Staploy file containing a single target operation:

```
target "production" {
   workers = ["worker-01"]

   set_active "my-app" {
      version = "1.0.0"
   }
}
```

This configuration selects `worker-01` and activates version `1.0.0` of `my-app`.

16.2 Multi-architecture Build

A build may contain architecture-specific targets together with files shared by all architectures.

```
build "my-app" {
   output_dir = "./build"

   executable = [
   "my-app",
   "helper"
   ]

   share {
      path = "./resources"
   }

   x86_64 {
      path = "./bin/x86_64"
   }

   aarch64 {
      path = "./bin/aarch64"
   }

   riscv64 {
      path = "./bin/riscv64"
   }
}
```

The `share` target contains files common to every architecture. Each architecture-specific target supplies the files applicable to that architecture. The values of `executable` identify executable file names contained in the resulting artifact.

16.3 Worker Alias

A worker alias can be used to give a reusable name to a group of worker references.

```
alias {
   worker "production" {
      workers = [
      "worker-01",
      "worker-02",
      "worker-03"
      ]
   }
}

target "deploy-production" {
   workers = ["alias:production"]

   deploy "my-app" {
      version = "1.0.0"
   }
}
```

The `alias:production` expression resolves to the workers defined by the `production` worker alias. Worker aliases are useful when the same set of workers is referenced by multiple targets.

16.4 App Alias

Application aliases can be used to provide reusable application references and versions.

```
alias {
   app "release" {
      name = "my-app"
      version = "1.0.0"
   }
}
```

```
target "production" {
   workers = ["group:production"]

   deploy "alias:release" {}
}
```

The `alias:release` expression resolves to the application and version specified by the alias. This allows the application version used by multiple operations to be changed in one place.

16.5 Build and Upload

A build artifact can be produced and subsequently uploaded to the local Staploy server.

```
build "my-app" {
   output_dir = "./build"
   version = "1.0.0"

   executable = [
   "my-app"
   ]

   x86_64 {
      path = "./bin/x86_64"
   }

   aarch64 {
      path = "./bin/aarch64"
   }
}

manage "my-app" {
   upload {
      path = "./build/my-app_1.0.0.tar"
   }
}
```

The `build` block produces the application artifact in the specified output directory. The `upload` operation then registers the locally produced package with the local Staploy server. The upload operation does not send the package to a worker.

16.6 Build, Upload, and Deploy

The following example combines building, uploading, and deployment into a single Staploy file.

```
build "my-app" {
   output_dir = "./build"
   version = "1.0.0"

   executable = [
   "my-app"
   ]

   share {
      path = "./resources"
   }

   x86_64 {
      path = "./bin/x86_64"
   }

   aarch64 {
      path = "./bin/aarch64"
   }
}

manage "my-app" {
   upload {
      path = "./build/my-app_1.0.0.tar"
   }
}

target "production" {
   workers = ["group:production"]

   deploy "my-app" {
      version = "1.0.0"

      pre_deploy  = "systemctl stop my-app"
      post_deploy = "systemctl start my-app"
   }
}
```

This configuration represents a complete local deployment workflow.

First, the application is built for the requested architectures. The resulting package is then uploaded to the local Staploy server. Finally, the specified application version is deployed to all workers selected by `group:production`. During deployment, the `pre_deploy` hook is executed before the deployment operation and the `post_deploy` hook is executed after it. The build and upload stages operate on the local environment and Staploy server, while the target stage performs operations on the selected workers.

16.7 Complete Deployment Example

The following example demonstrates a complete deployment workflow using worker aliases, application aliases, worker filtering, shell-based values, multi-architecture builds, package upload, and target operations.

```
alias {
   worker "zmx-deploy" {
      workers = ["group:all"]

      where {
         arch = ["x86_64", "arm", "aarch64", "mipsel"]
      }
   }

   app "zmx-build" {
      name    = "zmx"
      version = "shell:git describe --tags --abbrev=0"
   }
}

build "alias:zmx-build" {
   output_dir = "zig-out"
   executable = ["zmx"]

   envs        = ["ZIG_PATH=/home/$USER/.local/share/zig/0.16.0"]
   lib_version = "shell:echo zig $($(echo $ZIG_PATH)/zig version)"
   pre_build   = "git pull --no-edit"

   x86_64 {
      pre_build = <<-EOF
         $ZIG_PATH/zig build \
            -Doptimize=ReleaseSmall \
            -Dtarget=x86_64-linux-musl \
            --prefix ./zig-out/amd64
      EOF
      path = "zig-out/amd64/bin"
   }

   arm {
      pre_build = <<-EOF
         $ZIG_PATH/zig build \
            -Doptimize=ReleaseSmall \
            -Dtarget=arm-linux-musleabihf \
            --prefix ./zig-out/arm
      EOF
      path = "zig-out/arm/bin"
   }
```

```
   aarch64 {
      pre_build = <<-EOF
         $ZIG_PATH/zig build \
            -Doptimize=ReleaseSmall \
            -Dtarget=aarch64-linux-musl \
            --prefix ./zig-out/aarch64
      EOF
      path = "zig-out/aarch64/bin"
   }

   mipsel {
      pre_build = <<-EOF
         $ZIG_PATH/zig build \
            -Doptimize=ReleaseSmall \
            -Dtarget=mipsel-linux-musleabihf \
            --prefix ./zig-out/mipsel
      EOF
      path = "zig-out/mipsel/bin"
   }
}

configure {
   address = "192.168.50.194"
   port = 18090
   enforce_uuid = false
}

manage "alias:zmx-build" {
   upload {}
}

target "upload_zmx" {
   workers = ["alias:zmx-deploy"]

   unset "alias:zmx-build" {
      pre_deploy = "zmx k"
   }

   deploy "alias:zmx-build" {
      post_deploy = "zmx v"
   }
}
```

The example defines two aliases. The `zmx-deploy` worker alias initially selects all workers through the `group:all` expression and then restricts the resulting worker set to the architectures supported by the build.

The `zmx-build` application alias identifies the application as `zmx`. Its version is obtained dynamically using a `shell:` expression, which retrieves the latest Git tag. The `build` block uses the application alias and produces architecture-specific artifacts for `x86_64`, `arm`, `aarch64`, and `mipsel`. Each target invokes Zig with an architecture-specific compilation target and places the resulting executable in its corresponding output directory. The `lib_version` attribute demonstrates another use of the `shell:` expression. Its value is generated by executing a command in the build environment. The `manage` block uploads the resulting package to the configured local Staploy server.

Finally, the `upload_zmx` target performs the deployment. The `unset` operation first removes the currently active version from each selected worker. Its `pre_deploy` hook invokes the application's command to terminate or otherwise disable the currently running instance. The subsequent `deploy` operation pushes the newly built version and activates it. Its `post_deploy` hook invokes the application again after activation. This example demonstrates how aliases, resolution, build processing, package management, worker filtering, and target execution can be combined into a single deployment workflow.